HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

TIỂU LUẬN: PHÁT TRIỂN PHẦN MỀM HƯỚNG DỊCH VỤ

Đề tài: Xây dựng dịch vụ web theo kiến trúc REST về Quản lý sinh viên bằng JAX-RS trên nền tảng NetBeans và GlassFish, sử dụng file XML để lưu trữ dữ liệu.

Tên học phần: Phát triển phần mềm hướng dịch vụ

Nhóm (lớp học phần): INT1448.N1

Sinh viên tham gia thực hiện:

1. Nguyễn Văn A	Mã số: B24DTCN000	Lớp: D24TXCN00-B
2. Nguyễn Văn B	Mã số: B24DTCN001	Lớp: D24TXCN00-B
3. Nguyễn Văn C	Mã số: B24DTCN002	Lớp: D24TXCN00-B
4. Nguyễn Văn D	Mã số: B24DTCN003	Lớp: D24TXCN00-B
5. Nguyễn Văn E	Mã số: B24DTCN004	Lớp: D24TXCN00-B

Giáo viên hướng dẫn: ThS. Phạm Quang Hiếu

HÀ NỘI, 2025

PHÂN CÔNG NHIỆM VỤ THÀNH VIÊN NHÓM DỰ ÁN

STT	Họ và tên	Nhiệm vụ
1	**Nguyễn Văn A** Vai trò: Trưởng nhóm	1. Quản lý chung & Tài liệu hóa: Tổng hợp, kiểm tra, định dạng cuối cùng cho tiểu luận. 2. Thiết lập & Logic nghiệp vụ: * Tạo Project JAX-RS và cấu trúc thư mục. * Tạo Model SinhVien.java (Model Class). * Tạo lớp xử lý dữ liệu SinhVienService.java (Logic đọc/truy vấn dữ liệu từ XML).
2	**Nguyễn Văn B** Vai trò: Thành viên	Phát triển Resource (API Endpoint): * Tạo SinhVienResource.java (Định nghĩa các API GET bằng JAX-RS: lấy tất cả, tìm theo mã số, tìm theo lớp). * Khai báo RestApplication.java (Định nghĩa API base path /api).
3	**Nguyễn Văn C** Vai trò: Thành viên	Môi trường & Dữ liệu: * Cài đặt/Cấu hình NetBeans IDE, JDK và GlassFish Server. * Tạo file dữ liệu sinhvien.xml theo cấu trúc đã định. * Nghiên cứu DOM Parser và JSON Provider (Phần lý thuyết).
4	**Nguyễn Văn D** Vai trò: Thành viên	1. Lý thuyết: Viết và trình bày phần Giới thiệu chung (Mở đầu, Khái niệm REST, API REST). 2. Công cụ: Viết phần Công nghệ và Công cụ sử dụng (JAX-RS, NetBeans, GlassFish...) 3. Kiểm thử (Client): Thực hiện triển khai lên GlassFish và kiểm tra dịch vụ bằng Trình duyệt và Postman (Chụp ảnh/dán kết quả).
5	**Nguyễn Văn E** Vai trò: Thành viên	1. Đánh giá kết quả: Tổng kết Ưu điểm và phân tích; Hạn chế / Vấn đề đã gặp & Giải pháp trong quá trình thực hiện. 2. Hướng mở rộng: Xây dựng chi tiết nội dung Hướng mở rộng (Chuyển CSDL, CRUD, Bảo mật, Test, Caching, Client hiện đại). 3. Kết luận và Tài liệu tham khảo: Viết Kết luận và liệt kê Tài liệu tham khảo.

MỤC LỤC

I. TÊN ĐỀ TÀI 4
II. GIỚI THIỆU CHUNG 4
2.1 Mở đầu 4
2.2 Các khái niệm và công nghệ nền tảng của RESTful Web Service 4
2.2.1 REST 4
2.2.2 API REST 5
2.2.3 Công nghệ và công cụ sử dụng trong dự án (project) 6
III. THỰC HÀNH XÂY DỰNG DỊCH VỤ WEB 8
3.1. Cấu trúc cây thư mục tổng thể của dự án 8
3.2. Các bước triển khai thực hiện dự án 8
3.2.1 Tạo project JAX-RS (NetBeans) 8
3.2.2 Tạo file dữ liệu `sinhvien.xml` 8
3.2.3 Tạo model `SinhVien.java` 9
3.2.4 Tạo lớp xử lý dữ liệu `SinhVienService.java` 10
3.2.5 Tạo resource JAX-RS `SinhVienResource.java` 12
3.2.6 Khai báo Application 13
3.2.7 Triển khai lên GlassFish và kiểm tra dịch vụ 14
IV. ĐÁNH GIÁ KẾT QUẢ VÀ HƯỚNG MỞ RỘNG 18
4.1 Đánh giá kết quả 18
4.1.1 Ưu điểm: 18
4.1.2 Hạn chế / vấn đề đã gặp & giải pháp: 18
4.2 Hướng mở rộng 18
KẾT LUẬN 19
TÀI LIỆU THAM KHẢO 20

I. TÊN ĐỀ TÀI

Xây dựng dịch vụ web theo kiến trúc REST về Quản lý sinh viên bằng JAX-RS trên NetBeans và GlassFish, sử dụng file XML để lưu trữ dữ liệu.

II. GIỚI THIỆU CHUNG

2.1 Mở đầu

- Trong môi trường học tập, việc quản lý thông tin sinh viên (mã số, họ tên, giới tính, ngày sinh, lớp) là một nhu cầu rất cơ bản. Đề tài này này hướng dẫn cách xây dựng một dịch vụ web RESTful đơn giản để:

+ Cung cấp dữ liệu sinh viên,

+ Cho phép tìm kiếm/truy vấn theo yêu cầu,

+ Và dễ dàng mở rộng khi cần.

- Trong phạm vi bài học, dữ liệu ban đầu được lưu trữ trong file `sinhvien.xml` để đơn giản hoá việc thao tác (phù hợp cho tiểu luận, bài tập hoặc ứng dụng nhỏ). Về sau, nếu triển khai trong thực tế, có thể thay thế bằng cơ sở dữ liệu quan hệ.

- Mục tiêu của bài tập:

+ Xây dựng và thiết kế RESTful API với cú pháp rõ ràng, dễ sử dụng, đảm bảo mỗi tài nguyên (resource) được định nghĩa và truy cập hợp lý.

+ Làm quen và thực hành với các công nghệ nền tảng quan trọng như JAX-RS, DOM Parser, JSON Provider trong môi trường Java EE/Jakarta EE.

+ Triển khai trực tiếp trên NetBeans IDE kết hợp GlassFish Server để rèn kỹ năng phát triển, triển khai và vận hành dịch vụ web.

+ Đóng vai trò client để gọi, kiểm tra và đánh giá API, qua đó hiểu trọn vẹn cả hai phía: xây dựng dịch vụ và sử dụng dịch vụ.

2.2 Các khái niệm và công nghệ nền tảng của RESTful Web Service

2.2.1 REST

- REST (Representational State Transfer) là một kiểu kiến trúc phần mềm cho dịch vụ web, tập trung vào resource (tài nguyên). Mỗi tài nguyên (ví dụ: sinh viên, lớp học, môn học) sẽ được định danh duy nhất bằng một URI.

- Nguyên tắc chính:

+ Stateless (phi trạng thái): Mỗi yêu cầu (request) từ client (ứng dụng hoặc người dùng gửi yêu cầu đến dịch vụ, ví dụ: trình duyệt web, Postman, ứng dụng di động) phải chứa đầy đủ thông tin để server (máy chủ cung cấp và xử lý dịch vụ, nơi triển khai API hoặc ứng dụng web) có thể hiểu và xử lý. Server không lưu lại trạng thái phiên làm việc trước đó của client.

+ Uniform Interface (giao diện thống nhất): Tất cả resource đều được truy cập theo một cách giống nhau qua HTTP (giao thức truyền tải siêu văn bản) và địa chỉ URI (địa chỉ duy nhất định danh cho từng tài nguyên).

- Sử dụng các phương thức trong giao thức HTTP:

+ GET: Lấy thông tin tài nguyên.

+ POST: Thêm mới tài nguyên.

+ PUT: Cập nhật tài nguyên.

+ DELETE: Xoá tài nguyên.

- Representation (biểu diễn dữ liệu): Một resource có thể được trả về dưới nhiều dạng khác nhau, phổ biến là JSON (JavaScript Object Notation: gọn nhẹ, dễ đọc, thường dùng trong các ứng dụng web và mobile.) hoặc XML (eXtensible Markup Language: cấu trúc dạng thẻ, dễ mở rộng, thường dùng trong các hệ thống cũ hoặc tích hợp với SOAP).

Ví dụ:

- GET /api/sinhvien/sv/SV001

Kết quả trả về dữ liệu Json

```
{
  "maSo": "SV001",
  "hoTen": "Nguyen Van A",
  "gioiTinh": "Nam",
  "ngaySinh": "2003-05-10",
  "lop": "CTK45A"
}
```

Kết quả trả về dữ liệu XML

```
<SinhVien>
  <MaSo>SV001</MaSo>
  <HoTen>Nguyen Van A</HoTen>
  <Lop>CTK45A</Lop>
</SinhVien>
```

- GET api/sinhvien/lop/CTK45B

```
{
  "maSo": "SV002",
  "hoTen": "Tran Thi B",
  "gioiTinh": "Nữ",
  "ngaySinh": "2003-08-21",
  "lop": "CTK45B"
},
{
  "maSo": "SV005",
  "hoTen": "Do Van E",
  "gioiTinh": "Nam",
  "ngaySinh": "2002-11-29",
  "lop": "CTK45B"
}
```

2.2.2 API REST

API REST (RESTful API) là một giao diện lập trình ứng dụng (Application Programming Interface) được xây dựng dựa trên nguyên tắc REST

- API là "cầu nối" cho phép các phần mềm, ứng dụng khác nhau nói chuyện với nhau.
- REST là một kiến trúc thiết kế dùng HTTP (giao thức web) để trao đổi dữ liệu.

2.2.3 Công nghệ và công cụ sử dụng trong dự án (project)

1. Java EE / Jakarta EE (GlassFish Application Server)

- Là nền tảng để xây dựng và chạy ứng dụng web Java.
- Trong project này, sử dụng GlassFish Server 5.1 làm application server (máy chủ ứng dụng, nơi chạy và quản lý các ứng dụng web). Nó hoạt động như một container (môi trường chứa và cung cấp các dịch vụ nền tảng cho ứng dụng, ví dụ: Servlet, JSP, JAX-RS) để triển khai và vận hành dịch vụ web
- GlassFish hỗ trợ đầy đủ các API chuẩn như JAX-RS, JAX-WS, Servlet, JSP, EJB.

2. JAX-RS (Java API for RESTful Web Services)

- Là chuẩn trong Java EE/Jakarta EE dùng để xây dựng các dịch vụ web theo mô hình RESTful.
- Jersey là thư viện triển khai (implementation) phổ biến nhất, và đã được tích hợp sẵn trong GlassFish.
- Khi lập trình với JAX-RS, ta chỉ cần dùng các ghi chú đặc biệt (annotation) để định nghĩa REST API:

`@Path` → Chỉ ra đường dẫn (endpoint) cho resource.

`@GET`, `@POST`, `@PUT`, `@DELETE` → Tương ứng với các phương thức HTTP.

`@Produces`, `@Consumes` → Quy định dữ liệu trả về hoặc dữ liệu nhận vào (thường là JSON hoặc XML).

3. NetBeans IDE (Phiên bản 21)

- Là công cụ lập trình tích hợp (IDE) hỗ trợ phát triển ứng dụng Java EE/Jakarta EE.
- Giúp quản lý dự án web dễ dàng: tự động tạo cấu trúc thư mục, biên dịch (build) và triển khai (deploy) ứng dụng trực tiếp lên GlassFish Server.
- Hỗ trợ gỡ lỗi (debug) và kiểm thử (test) REST/SOAP Web Services ngay trong IDE, không cần dùng thêm công cụ bên ngoài.

4. GlassFish5 (Java EE 7 hoặc Java EE 8):

Là một máy chủ ứng dụng (application server) mã nguồn mở dành cho nền tảng Java EE (nay là Jakarta EE). Nó được thiết kế để cung cấp một môi trường hoàn chỉnh và mạnh mẽ để triển khai, quản lý và chạy các ứng dụng doanh nghiệp được xây dựng bằng công nghệ Java.

5. JDK (Java Development Kit Java 8/17)

- JDK *(phiên bản 8):* là bộ công cụ phát triển phần mềm bằng ngôn ngữ Java, nhằm tích hợp vào GlassFish5.
- JDK *(phiên bản 17):* là bộ công cụ phát triển phần mềm bằng ngôn ngữ Java, nhằm tích hợp vào NetBeans.

6. DOM Parser (Document Object Model Parser)

- Là thư viện chuẩn của Java (gồm các package như `javax.xml.parsers`, `org.w3c.dom`).

- Dùng để đọc và ghi file XML (trong project này là sinhvien.xml).

- Cách hoạt động: nạp (parse) file XML → chuyển nội dung thành cấu trúc cây DOM trong bộ nhớ → lập trình viên có thể thêm, sửa, xóa các nút (node) → ghi lại thành file XML mới.

- Cách này đơn giản, phù hợp với dữ liệu nhỏ như danh sách sinh viên, nhưng không tối ưu nếu dữ liệu quá lớn.

7. JSON Provider

JSON Provider giống như một "phiên dịch viên"**,** giúp đối tượng Java (`SinhVien`) có thể "nói chuyện" bằng JSON để client (ứng dụng web, Postman, trình duyệt) hiểu được.

8. Client

Để gọi và kiểm thử REST API, ta có thể dùng nhiều cách khác nhau:

- Jersey Client API: thư viện Java giúp tạo và gửi HTTP request/response dễ dàng.

- HttpURLConnection (có sẵn trong JDK): cách làm thủ công hơn, nhưng không cần thêm thư viện ngoài.

- Postman: công cụ có giao diện trực quan, phổ biến, rất tiện để gửi request và xem kết quả trả về.

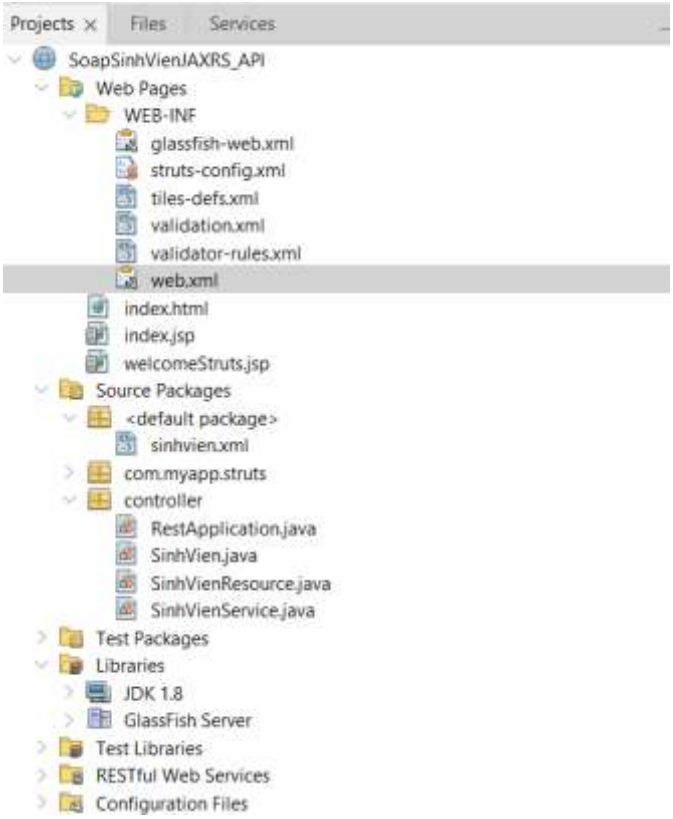
- SoapUI: công cụ chuyên dụng, mạnh mẽ, hỗ trợ kiểm thử cả dịch vụ REST lẫn SOAP.

- Curl**:** lệnh chạy trong terminal/command line, đơn giản nhưng rất hữu ích để test nhanh.

Tuy nhiên, trong dự án này sử dụng Postman để thử REST API

III. THỰC HÀNH XÂY DỰNG DỊCH VỤ WEB

3.1. Cấu trúc cây thư mục tổng thể của dự án



3.2. Các bước triển khai thực hiện dự án

3.2.1 Tạo project JAX-RS (NetBeans)

1. New Project → Java Web → Web Application → đặt tên `SoapSinhVienJAXRS_API`.

2. Chọn server: GlassFish; Java EE version: 7/8 tương thích.

3. Tạo cấu trúc package `controller`.

3.2.2 Tạo file dữ liệu `sinhvien.xml`

- Đây là nguồn dữ liệu chính (data source) của toàn bộ dịch vụ web.

- File này lưu danh sách sinh viên dưới dạng XML (bao gồm: mã số, họ tên, giới tính, ngày sinh, lớp).

- Khi REST API chạy, các lớp Java (ví dụ `SinhVienService`) sẽ đọc file sinhvien.xml, parse thành đối tượng `SinhVien`, sau đó trả về dữ liệu dưới dạng JSON cho client.

- Các chức năng như lấy toàn bộ sinh viên, tìm sinh viên theo mã số, tìm sinh viên theo lớp đều dựa trên dữ liệu gốc trong file này.

```
<?xml version="1.0" encoding="UTF-8"?>
<DanhSachSinhVien>
    <SinhVien>
        <MaSo>SV001</MaSo>
        <HoTen>Nguyen Van A</HoTen>
```

```
        <GioiTinh>Nam</GioiTinh>
        <NgaySinh>2003-05-10</NgaySinh>
        <Lop>CTK45A</Lop>
    </SinhVien>
    <SinhVien>
        <MaSo>SV002</MaSo>
        <HoTen>Tran Thi B</HoTen>
        <GioiTinh>Nữ</GioiTinh>
        <NgaySinh>2003-08-21</NgaySinh>
        <Lop>CTK45B</Lop>
    </SinhVien>
    <SinhVien>
        <MaSo>SV003</MaSo>
        <HoTen>Le Van C</HoTen>
        <GioiTinh>Nam</GioiTinh>
        <NgaySinh>2002-12-15</NgaySinh>
        <Lop>CTK45A</Lop>
    </SinhVien>
    <SinhVien>
        <MaSo>SV004</MaSo>
        <HoTen>Pham Thi D</HoTen>
        <GioiTinh>Nữ</GioiTinh>
        <NgaySinh>2003-04-02</NgaySinh>
        <Lop>CTK45C</Lop>
    </SinhVien>
    <SinhVien>
        <MaSo>SV005</MaSo>
        <HoTen>Do Van E</HoTen>
        <GioiTinh>Nam</GioiTinh>
        <NgaySinh>2002-11-29</NgaySinh>
        <Lop>CTK45B</Lop>
    </SinhVien>
</DanhSachSinhVien>
```

3.2.3 Tạo model `SinhVien.java`

- Đây là lớp mô hình (Model class) đại diện cho một sinh viên.
- Nó định nghĩa các thuộc tính (fields): `maSo`, `hoTen`, `gioiTinh`, `ngaySinh`, `lop`.
- Cung cấp constructor (hàm khởi tạo), getter (hàm lấy giá trị của thuộc tình) và setter (hàm gán giá trị của thuộc tính) để tạo và thao tác với đối tượng sinh viên.

- Khi dịch vụ đọc dữ liệu từ `sinhvien.xml`, mỗi node `<SinhVien>` trong file XML sẽ được chuyển thành một đối tượng `SinhVien`.

- Khi trả dữ liệu ra ngoài qua REST API, các đối tượng `SinhVien` này được serialize sang JSON (nhờ JSON Provider).

- Đây chính là cầu nối giữa dữ liệu XML gốc và API REST trả ra JSON.

```
package controller;

public class SinhVien {
    private String maSo;
    private String hoTen;
    private String gioiTinh;
    private String ngaySinh;
    private String lop;

    public SinhVien() {}

    public SinhVien(String maSo, String hoTen, String gioiTinh, String
ngaySinh, String lop) {
        this.maSo = maSo;
        this.hoTen = hoTen;
        this.gioiTinh = gioiTinh;
        this.ngaySinh = ngaySinh;
        this.lop = lop;
    }

    public String getMaSo() { return maSo; }
    public void setMaSo(String maSo) { this.maSo = maSo; }

    public String getHoTen() { return hoTen; }
    public void setHoTen(String hoTen) { this.hoTen = hoTen; }

    public String getGioiTinh() { return gioiTinh; }
    public void setGioiTinh(String gioiTinh) { this.gioiTinh = gioiTinh; }

    public String getNgaySinh() { return ngaySinh; }
    public void setNgaySinh(String ngaySinh) { this.ngaySinh = ngaySinh; }

    public String getLop() { return lop; }
    public void setLop(String lop) { this.lop = lop; }
}
```

3.2.4 Tạo lớp xử lý dữ liệu `SinhVienService.java`

- Đây là lớp dịch vụ (Service class), chuyên xử lý logic nghiệp vụ liên quan đến dữ liệu sinh viên.

- Cụ thể:

+ Đọc dữ liệu từ file `sinhvien.xml` bằng DOM Parser và chuyển thành danh sách các đối tượng `SinhVien`.

+ Cung cấp hàm `getAllSinhVien()` để lấy toàn bộ danh sách sinh viên.

+ Cung cấp hàm `findByMaSo(maSo)` để tìm sinh viên theo mã số.

+ Cung cấp hàm `findByLop(maLop)` để tìm danh sách sinh viên theo lớp.

- Đây chính là lớp trung gian:

+ Input: dữ liệu gốc trong file XML.

+ Xử lý: đọc, duyệt, lọc theo yêu cầu.

+ Output: trả về danh sách hoặc đối tượng `SinhVien` cho lớp `SinhVienResource` (REST API).

```
package controller;
```

```
import java.io.InputStream;
import java.util.ArrayList;
import java.util.List;
import javax.xml.parsers.DocumentBuilderFactory;
import org.w3c.dom.*;
import java.io.File;
import javax.xml.transform.*;
import javax.xml.transform.dom.DOMSource;
import javax.xml.transform.stream.StreamResult;

public class SinhVienService {
    //Lấy toàn bộ danh sách sinh viên
    public static List<SinhVien> getAllSinhVien() {
        List<SinhVien> ds = new ArrayList<>();
        try {

            //String is1 = System.getProperty("catalina.base"); // thư mục
Tomcat
            //String is = is1 + "/WEB-INF/sinhvien.xml";
            InputStream is =
SinhVienService.class.getClassLoader().getResourceAsStream("sinhvien.xml");
            Document doc =
DocumentBuilderFactory.newInstance().newDocumentBuilder().parse(is);
            doc.getDocumentElement().normalize();
            NodeList nList = doc.getElementsByTagName("SinhVien");

            for (int temp = 0; temp < nList.getLength(); temp++) {
                Node nNode = nList.item(temp);
                if (nNode.getNodeType() == Node.ELEMENT_NODE) {
                    Element eElement = (Element) nNode;

                    SinhVien sv = new SinhVien(

eElement.getElementsByTagName("MaSo").item(0).getTextContent(),

eElement.getElementsByTagName("HoTen").item(0).getTextContent(),

eElement.getElementsByTagName("GioiTinh").item(0).getTextContent(),

eElement.getElementsByTagName("NgaySinh").item(0).getTextContent(),

eElement.getElementsByTagName("Lop").item(0).getTextContent()
                    );
                    ds.add(sv);
                }
            }
```

```
        } catch (Exception e) {
            e.printStackTrace();
        }
        return ds;
    }
    //Lấy thông tin sinh viên theo mã số
    public static SinhVien findByMaSo(String maSo) {
        for (SinhVien sv : getAllSinhVien()) {
            if (sv.getMaSo().equalsIgnoreCase(maSo)) {
                return sv;
            }
        }
        return null;
    }
    //Tìm danh sách sinh viên theo lớp
    public static List<SinhVien> findByLop(String maLop) {
        List<SinhVien> dsTheoLop = new ArrayList<>();
        for (SinhVien sv : getAllSinhVien()) {
            if (sv.getLop().equalsIgnoreCase(maLop)) {
                dsTheoLop.add(sv);
            }
        }
        return dsTheoLop;
    }

}
```

3.2.5 Tạo resource JAX-RS `SinhVienResource.java`

- Đây là lớp RESTful Resource, chịu trách nhiệm định nghĩa các API endpoint để client (ứng dụng hoặc người dùng) có thể gọi đến.

- Sử dụng annotation JAX-RS như:

`@Path` (xác định URL cho API),

`@GET`, `@POST`, `@PUT`, `@DELETE` (chỉ rõ phương thức HTTP),

`@Produces` và `@Consumes` (quy định dữ liệu trả về/nhận vào, thường là JSON hoặc XML).

- Các phương thức trong lớp này gọi đến `SinhVienService` để thực hiện xử lý, sau đó trả kết quả cho client.

- Cụ thể là:

+ API `/sinhvien/` → gọi `service.getAllSinhVien()` → trả về toàn bộ danh sách sinh viên.

+ API `/sinhvien/sv/{maSo}` → gọi `service.findByMaSo(maSo)` → trả về sinh viên theo mã số.

+ API `/sinhvien/lop/lop/{maLop}` → gọi `service.findByLop(maLop)` → trả về danh sách sinh viên theo lớp.

```
package controller;

import javax.ws.rs.*;
import javax.ws.rs.core.MediaType;
import java.util.List;

@Path("/sinhvien")
public class SinhVienResource {

    @GET
    @Produces(MediaType.APPLICATION_JSON)
    public List<SinhVien> getAllSinhVien() {
        return SinhVienService.getAllSinhVien();
    }

    @GET
    @Path("/sv/{maSo}")
    @Produces(MediaType.APPLICATION_JSON)
    public SinhVien getSinhVienByMaSo(@PathParam("maSo") String maSo) {
        return SinhVienService.findByMaSo(maSo);
    }

    @GET
    @Path("/lop/{maLop}")
    @Produces(MediaType.APPLICATION_JSON)
    public List<SinhVien> getSinhVienByLop(@PathParam("maLop") String maLop) {
        return SinhVienService.findByLop(maLop);
    }

}
```

Lưu ý: bạn đặt đường `/sv/{maSo}` thay vì `/{maSo}` để tránh xung đột với các route khác — đó là một lựa chọn rõ ràng.

3.2.6 Khai báo Application

- Tạo file RestApplication.java để bật JAX-RS (Java API cho REST).

- Trong file này, ta quy định API base path là /api. Điều đó có nghĩa là tất cả các endpoint (đường dẫn API cụ thể) của dịch vụ sẽ bắt đầu bằng /api/sinhvien/....

Cụ thể:

`/api/sinhvien/` → lấy toàn bộ danh sách sinh viên.

`/api/sinhvien/sv/{maSo}` → lấy thông tin một sinh viên theo mã số.

`/api/sinhvien/lop/{maLop}` → lấy danh sách sinh viên theo lớp

```
package controller;

import javax.ws.rs.ApplicationPath;
import javax.ws.rs.core.Application;

@ApplicationPath("/api")
public class RestApplication extends Application {
    // Không cần code thêm
```

```
}
```

3.2.7 Triển khai lên GlassFish và kiểm tra dịch vụ

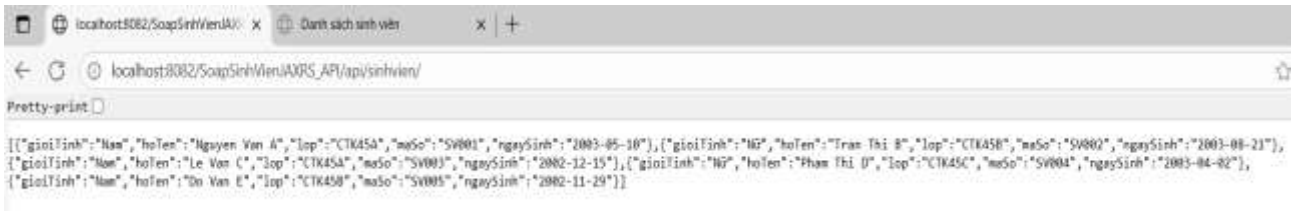
- Tiển khai dự án lên GlassFish.
- Kiểm tra bằng trình duyệt / Postman / curl:

❖ **Kiển tra bằng trình duyệt**

- Lấy toàn bộ danh sách sinh viên:

GET http://localhost:8082/SoapSinhVienJAXRS_API/api/sinhvien

- http://localhost:8082 là địa chỉ host (máy chủ cục bộ, cổng 8082).
- /SoapSinhVienJAXRS_API là tên ứng dụng web (web context).
- /api là đường dẫn cơ sở của API (được khai báo trong RestApplication.java).
- /sinhvien là đường dẫn đến tài nguyên - dịch vụ (định nghĩa trong @Path("sinhvien") ở SinhVienResource.java).

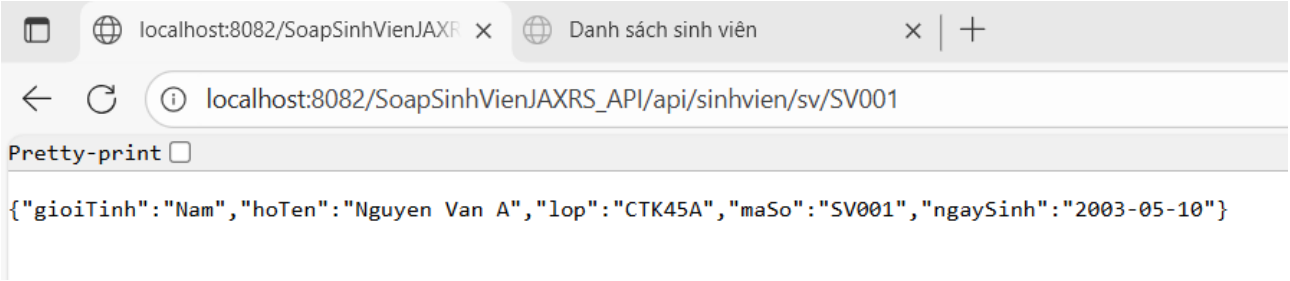



```
{
  "DanhSachSinhVien": [
    {
      "maSo": "SV001",
      "hoTen": "Nguyen Van A",
      "gioiTinh": "Nam",
      "ngaySinh": "2003-05-10",
      "lop": "CTK45A"
    },
    {
      "maSo": "SV002",
      "hoTen": "Tran Thi B",
      "gioiTinh": "Nữ",
      "ngaySinh": "2003-08-21",
      "lop": "CTK45B"
    },
    {
      "maSo": "SV003",
      "hoTen": "Le Van C",
      "gioiTinh": "Nam",
      "ngaySinh": "2002-12-15",
      "lop": "CTK45A"
    },
    {
      "maSo": "SV004",
```

```
      "hoTen": "Pham Thi D",
      "gioiTinh": "Nữ",
      "ngaySinh": "2003-04-02",
      "lop": "CTK45C"
    },
    {
      "maSo": "SV005",
      "hoTen": "Do Van E",
      "gioiTinh": "Nam",
      "ngaySinh": "2002-11-29",
      "lop": "CTK45B"
    }
  ]
}
```


- Lấy thông tin sinh viên theo mã số:

GET http://localhost:8082/SoapSinhVienJAXRS_API/api/sinhvien/sv/SV001




```
[
  {
    "maSo":"SV001",
    "hoTen":"Nguyen Van A",
    "gioiTinh":"Nam",
    "ngaySinh":"2003-05-10",
    "lop":"CTK45A"
  },
  ...
]
```


- Lấy danh sách sinh viên theo mã lớp:

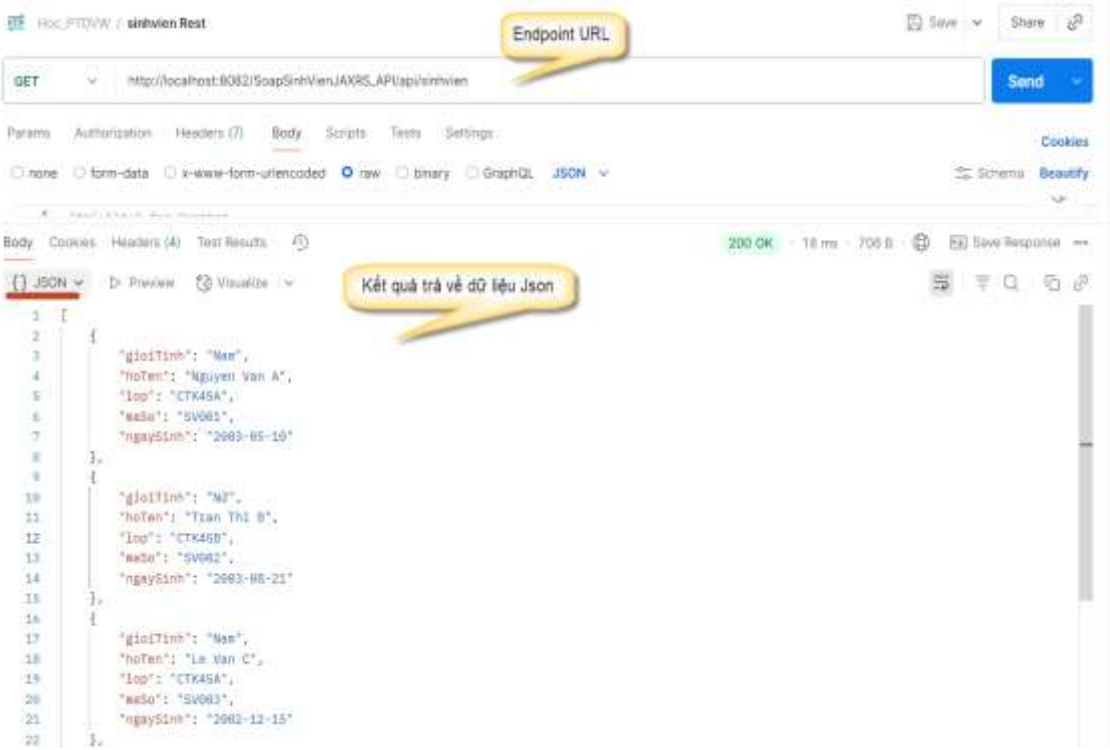
GET http://localhost:8082/SoapSinhVienJAXRS_API/api/sinhvien/lop/CTK45A

```
  [
    {
      "maSo": "SV002",
      "hoTen": "Tran Thi B",
      "gioiTinh": "Nữ",
      "ngaySinh": "2003-08-21",
```

```
    "lop": "CTK45B"
  },
  {
    "maSo": "SV005",
    "hoTen": "Do Van E",
    "gioiTinh": "Nam",
    "ngaySinh": "2002-11-29",
    "lop": "CTK45B"
  }
]
```


❖ Kiển tra bằng Postman

- Lấy toàn bộ danh sách sinh viên:

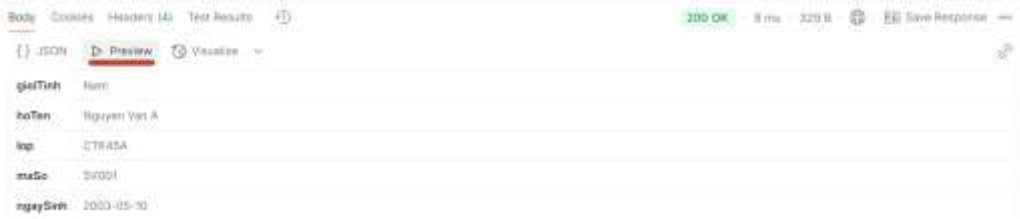


Xem kết quả chế độ Preview

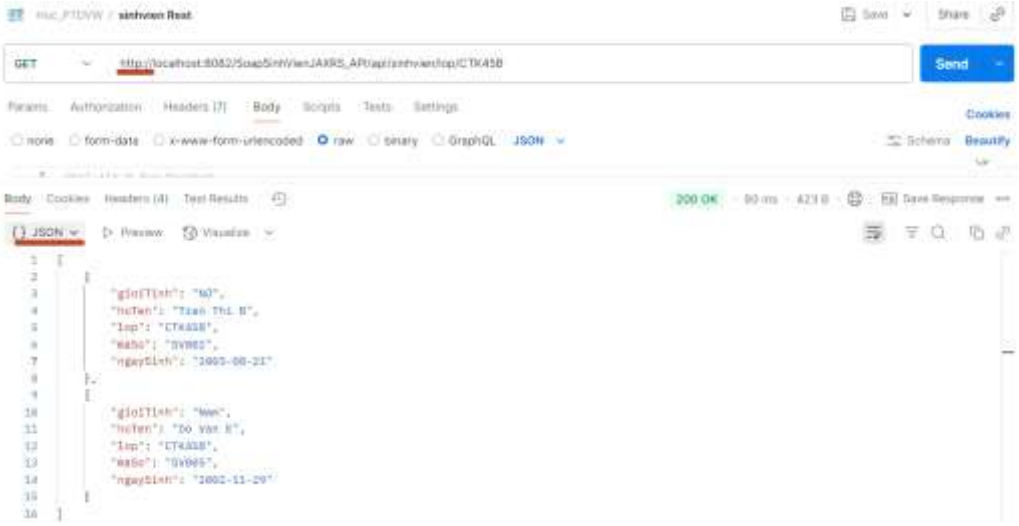
	gioiTinh	hoTen	lop	maSo	ngaySinh
0	Nam	Nguyen Van A	CTK45A	SV001	2003-05-10
1	Nữ	Tran Thi B	CTK45B	SV002	2003-08-21
2	Nam	Le Van C	CTK45A	SV003	2002-12-15
3	Nữ	Pham Thi D	CTK45C	SV004	2003-04-02
4	Nam	Do Van E	CTK45B	SV005	2002-11-29

- Lấy thông tin sinh viên theo mã số:

Xem kết quả chế độ Preview



- Lấy danh sách sinh viên theo mã lớp:



Xem kết quả chế độ Preview

IV. ĐÁNH GIÁ KẾT QUẢ VÀ HƯỚNG MỞ RỘNG

4.1 Đánh giá kết quả

4.1.1 Ưu điểm:

- Thiết kế rõ ràng, dễ hiểu: cấu trúc dự án gọn nhẹ, phù hợp với mục tiêu học tập, thực hành và trình diễn.

- API chuẩn REST: mỗi tài nguyên (resource) đều có URI hợp lý, giúp việc truy cập và sử dụng thống nhất, không rườm rà.

- Triển khai linh hoạt với JAX-RS: gọn nhẹ, dễ dàng mở rộng hoặc tích hợp thêm tính năng khi cần.

4.1.2 Hạn chế / vấn đề đã gặp & giải pháp:

- Ghi file XML trong WAR: khi triển khai (deploy) trên server, thường không ghi trực tiếp được.

Giải pháp: đặt file `sinhvien.xml` ở thư mục ngoài như `domain1/config` hoặc một thư mục `app_data` có quyền ghi, hoặc tốt hơn là chuyển sang dùng cơ sở dữ liệu.

- JSON Provider: không phải server nào cũng có sẵn JSON provider mặc định.

Giải pháp: bổ sung thư viện (dependency) như `jersey-media-json-binding` hoặc `jersey-media-moxy`, đồng thời dùng annotation như `@XmlRootElement` trên các POJO để hỗ trợ mapping.

- Truy cập đồng thời (Concurrency): nếu nhiều request cùng ghi file XML có thể dẫn đến lỗi hoặc hỏng dữ liệu.

Giải pháp: dùng đồng bộ hoá (`synchronized`) hoặc cơ chế file lock để tránh xung đột.

- Kiểm tra dữ liệu & xử lý lỗi (Validation & Error Handling): hiện tại chưa xử lý triệt để.

Giải pháp: cần trả về HTTP status code phù hợp (400, 404, 500...) kèm thông báo lỗi rõ ràng để client dễ xử lý.

- Bảo mật: demo chưa có cơ chế đăng nhập hay phân quyền.

Giải pháp: bổ sung xác thực (authentication) và phân quyền (authorization) nếu áp dụng vào môi trường thực tế.

4.2 Hướng mở rộng

1. Chuyển dữ liệu sang CSDL

Sử dụng MySQL hoặc PostgreSQL để thay thế file XML.

Ưu điểm: quản lý dữ liệu bền vững, truy vấn nhanh, hỗ trợ transaction.

2. Mở rộng chức năng CRUD + phân trang

Hiện mới có GET/POST, cần bổ sung đầy đủ PUT, DELETE.

Thêm phân trang, lọc (filter) và sắp xếp (sort) cho danh sách sinh viên.

3. Tích hợp OpenAPI/Swagger

Tự động sinh tài liệu API (swagger-ui), giúp developer khác dễ hiểu và dễ test API.

4. Viết test tự động

Sử dụng JUnit + REST-assured để viết unit test và integration test, đảm bảo API chạy đúng khi nâng cấp.

5. Xử lý đồng thời (Concurrency)

Nếu vẫn dùng file XML: cần cơ chế locking hoặc đồng bộ (synchronized).

Hoặc nâng cấp: dùng message queue + worker để xử lý ghi dữ liệu an toàn.

6. Caching cho API đọc nhiều

Áp dụng cache (ví dụ EHCache, Redis) cho các endpoint truy vấn nhiều để giảm tải server.

7. Bảo mật API

Bổ sung authentication/authorization.

Có thể dùng JWT hoặc OAuth2 nếu muốn triển khai trong môi trường thật.

8. Phát triển Client hiện đại

Xây dựng giao diện web dạng SPA (Single Page Application) bằng React hoặc Vue.

Gọi API trực tiếp, UI mượt và trải nghiệm tốt hơn JSP/Servlet.

9. DevOps & Triển khai

Đóng gói ứng dụng bằng Docker để dễ deploy.

Thiết lập CI/CD pipeline (GitHub Actions, GitLab CI) để tự động build, test, triển khai.

KẾT LUẬN

Dự án SoapSinhVienJAXRS_API là một trải nghiệm thực hành vô cùng bổ ích. Qua quá trình xây dựng, chúng em đã được làm quen và trực tiếp áp dụng những khái niệm quan trọng trong phát triển RESTful API:

- Hiểu cách tổ chức resource và ánh xạ URI để mỗi tài nguyên sinh viên có thể được truy cập rõ ràng, dễ hiểu.
- Biết cách đọc/ghi dữ liệu XML bằng DOM Parser trên dữ liệu sinhvien.xml.
- Thực hành triển khai dự án trên Netbeans cùng với GlassFish server

Kết quả đạt được không chỉ là một API chạy được, mà còn là sự rèn luyện kỹ năng tự học, tự gỡ lỗi, và nhìn thấy bức tranh tổng thể của một dịch vụ web.

Tuy nhiên, để dự án tiến xa hơn và tiệm cận ứng dụng thực tế, cần tiếp tục:

- Chuyển dữ liệu sang cơ sở dữ liệu (MySQL, PostgreSQL…) để tăng độ ổn định, bền vững và hỗ trợ truy vấn phức tạp.
- Bổ sung bảo mật (authentication/authorization) và cơ chế xử lý đồng thời để hệ thống không bị lỗi khi nhiều người dùng truy cập cùng lúc.
- Hoàn thiện triển khai: xử lý triệt để vấn đề ghi file, cấu hình JSON provider chuẩn, và nâng cấp code theo chuẩn sạch (clean code).

TÀI LIỆU THAM KHẢO

1. REST Web Services 10 – What Is JAX-RS? giải thích JAX-RS là gì, mối quan hệ với Jersey. YouTube

2. Building RESTful Web Services with JAX-RS hướng dẫn cách xây dựng dịch vụ RESTful bằng JAX-RS. YouTube

3. REST API with Java JAX-RS – Video Course: series nhiều phần, từ khởi tạo cho tới thực hành. YouTube

4. RESTful Web Services Tutorials | JAX RS Restful Injection: video về cách dùng `@PathParam`, injection trong JAX-RS. YouTube

5. Advanced JAX-RS (playlist): các chủ đề nâng cao như cấu hình phức tạp hơn. YouTube

6. Tạo ứng dụng Angular gọi RESTful Webservice (JAX-RS, Jersey): dùng để hiểu tương tác client ↔ server, dù là Angular nhưng phần REST backend sẽ giống. YouTube

7. Bài 21: Tìm hiểu về REST và RESTful API: có giải thích khái niệm REST trong ứng dụng thực tế (tiếng Việt) YouTube

8. RESTful Api: Bạn Đang Thiết Kế Sai Chuẩn REST (Fix Ngay) – video tiếng Việt nói về các nguyên tắc REST và lỗi hay gặp khi thiết kế REST API YouTube